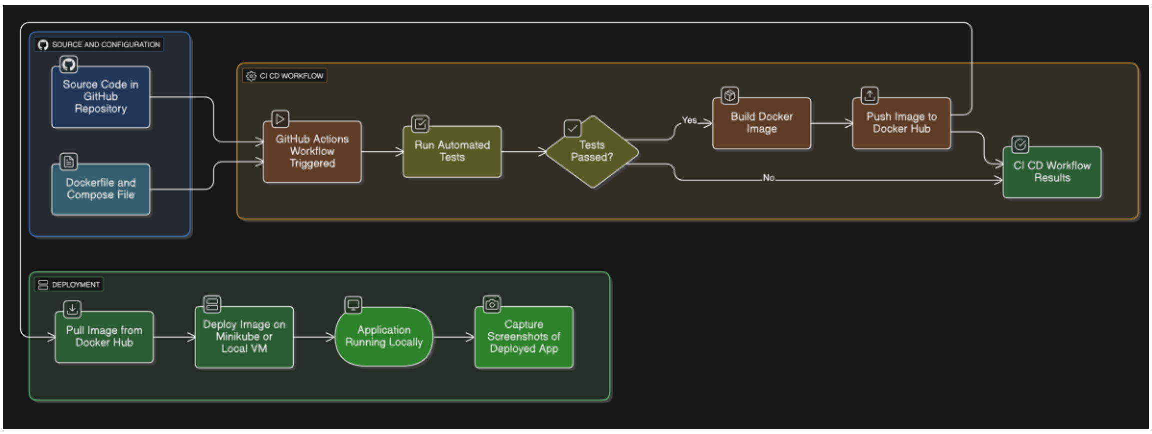
CI/CD Pipeline with GitHub Actions & Docker (Local Deployment)

Workflow:



1.Objective:

The goal of this project is to design and implement a CI/CD pipeline using GitHub Actions and Docker. The pipeline automates the process of building, testing, and deploying an application, without depending on cloud providers. Instead, deployment is performed locally using Minikube or a local VM.

This setup ensures:

- Automated builds and testing upon every code commit.
- Reliable and consistent Docker images published to Docker Hub.
- Local deployment and validation of the containerized app.

2. Tools & Technologies:

- GitHub Actions – for CI/CD automation.
- Docker – containerization platform for building and packaging the app.
- Docker Hub (Free) – remote registry for pushing and pulling images.
- Minikube / Local VM – for local deployment of the containerized app.
- docker-compose – for local testing before pushing images.

3. Project Workflow:

Step 1: Write Dockerfile & docker-compose.yml

- Created a Dockerfile that defines the application’s runtime environment.
- Wrote a docker-compose.yml file for local multi-service orchestration and testing.

Step 2: GitHub Actions Workflow

- Configured .github/workflows/ci-cd.yml.
- Workflow triggers:
  On push to main branch.
  On pull request for pre-merge validation.
  Stages included:
  1. Checkout code from repository.
  2. Run tests (e.g., unit tests).
  3. Build Docker image.
  4. Push image to Docker Hub using secrets (DOCKER_USERNAME, DOCKER_PASSWORD).

Step 3: Local Deployment

- Pulled Docker image from Docker Hub.
- Used Minikube or a local VM to run the image.

4. Results:

- Every code commit now automatically triggers the CI/CD pipeline.

- The application builds, passes tests, and pushes to Docker Hub without manual steps.

- Deployment is reproducible locally with minimal setup.

5.Conculsion:

- This project demonstrates how to implement a complete CI/CD pipeline without cloud infrastructure.
- By leveraging GitHub Actions and Docker Hub, developers can:
  Ensure consistent builds.
  Automate deployment workflows.